

Research Statement

Tamer Abdelaziz, Ph.D.

Postdoctoral Associate, New York University Abu Dhabi

Ph.D., National University of Singapore

tamer.m@nyu.edu • tamer-abdelaziz.github.io

The Problem I Work On

Blockchains now hold hundreds of billions of dollars in software-controlled assets, and when that software fails a single transaction can drain a protocol with no authority able to reverse it. The Gulf states are building on exactly this infrastructure, licensing virtual-asset businesses, piloting digital currencies, and tokenizing real-world assets. That makes its security a regional problem as much as a technical one.

My work follows one problem through the lifecycle of a blockchain system. I screen vulnerable code before deployment, measure where existing defenses fall short, extend detection to contracts no one has configured in advance, block malicious transactions at the moment of execution, and enforce regulatory rules on tokenized assets. Machine learning gives me the scale this needs; formal methods and protocol design give me results I can trust. The last two years changed what I think the goal should be. A detector tells you an attack is under way and leaves you to act in the seconds remaining. My recent work removes that gap, and it is the agenda I want to build a group around.

Detect: Teaching Neural Networks from Program Analyzers

Precise analyzers for smart contracts run too slowly to screen the volume of code reaching public blockchains; fast statistical detectors guess too often. My USENIX Security 2023 paper resolved this tension [1]. I used rigorous program analyzers as *teachers*: they label large training corpora, and deep neural networks learn to reproduce their judgments directly from EVM bytecode. The student models answer vulnerability queries orders of magnitude faster than their teachers, at an accuracy that makes screening at chain scale practical, and because they read bytecode they cover the majority of deployed contracts, whose source never appears on-chain. The work passed USENIX artifact evaluation, and the datasets and benchmarks I released with it are now used by other groups.

A follow-up paper asked the attacker's question instead: not whether a contract is vulnerable, but whether anyone can actually exploit it, which is the property that decides whether money moves [2]. My NUS thesis brought this detection work together [3].

Where Our Tools Fall Short

Building better defenses first meant finding out where the current ones fail. At MSR 2026 my co-authors and I studied smart contract security analyzers in practice [4], mapping the distance between what they claim to find and what real attacks exploit. That map still sets my targets, and it moved my attention to transactions: much of what analyzers miss is not visible in the contract's code and shows up only when someone attacks it.

A Detector That Needs to Know Nothing

That pushed the defense onto the live network. TxLens, published at IEEE ICBC 2026, screens pending Ethereum transactions at mempool latency, simulating each one and classifying the execution trace it produces [5]. Building it also showed me its ceiling. Every strong transaction-level detector, TxLens among them, needs something already in place before it can judge anything. Some rank a transaction against the contract's own history, which fails on quiet contracts and flags rare but legitimate transactions. Some recognize only a fixed catalogue of exploit classes, so

a novel attack is not just missed, it is reported as benign. Others must first be handed a protocol's contracts, ABIs, and storage layouts. A contract deployed yesterday offers none of this.

TxSentinel (under review) drops all three requirements [6]. It scores each transaction on its own, from its own execution trace, fixes no exploit classes, and reads only the standardized EVM trace schema, so it can judge a contract nobody has onboarded and an attack shape no taxonomy anticipated. Everything then rests on how a trace is represented, so I ran a comparison the field had skipped. I built three representations of the same traces: features engineered from the economics and control flow of an execution, frozen embeddings from a pretrained encoder, and embeddings I pretrained and contrastively fine-tuned myself. All three went through the same detectors under one training protocol, with nothing varying but the input. The engineered features won clearly over both learned encodings. I think that result matters beyond my own tool: for this problem, what a representation preserves counts for more than the capacity of the model reading it.

Confirmed exploit labels are scarce in practice, so I trained the system under that constraint rather than assuming a labeled corpus, then evaluated it on a benchmark of recent incidents I curated by hand and kept disjoint from training and model selection. TxSentinel outperforms both the academic state of the art and the commercial screening platforms CertiK Skynet and BlockSec Phalcon, catching attacks that both of them call benign. It carries to a second chain with one incremental training pass, and answers fast enough to sit in a live transaction path. It is also dual-use, since an attacker could query it to confirm an exploit works, so I will release it as a controlled-access service rather than as open weights. That decision was part of designing it.

Prevent: Making the Contract Refuse the Attack

Detection still leaves the outcome in the attacker's hands, and realising that changed my direction. A mempool defense observes a pending transaction, then races a mitigation against it. An attacker who submits through a private relay is never observed, and one who bids higher wins the race. The defense depends on the attacker's cooperation. Underneath this sits a question about what a blockchain signature actually certifies. Every deployed authorization protocol, EIP-712 and ERC-4337 and bridge quorums alike, records that somebody approved a call. None of them records whether running it is safe.

My most recent work introduces **Security Authorization**: a contract runs a sensitive function only after a trusted authority has analyzed that exact transaction under a stated policy and cryptographically approved it [7]. I realize it as the Security Authorization Framework (SAF, under review). An off-chain Authorization Service simulates the intended call, scores the resulting trace, and issues a short-lived Execution Permit only if the call passes policy; an on-chain Authorization Gateway verifies that permit before forwarding the call. What this changes is who controls the encounter. Under mempool monitoring the attacker decides whether the defense sees the call at all, and whether it can react in time. Under SAF neither choice is available, because getting the permit that execution requires means handing the exact call to the analyzer first. The remaining question is how good the analysis is, which I can measure and improve, rather than whether an attacker can route around me.

I built the evaluation to keep two very different guarantees apart, since the literature tends to run them together. What the Gateway enforces holds whatever the policy oracle concludes: no gated call executes without a permit, permits cannot be replayed or retargeted, and reentrancy fails structurally. A battery of protocol-level attacks against the authorization confirms this. What the oracle decides is a learned judgment, so I measured it separately against real attacks postdating its training data, and I diagnose the cases it misses rather than reporting only the ones it catches. I retrofitted the guard automatically onto a large corpus of historically exploited Ethereum and BNB Chain contracts, and a single unmodified Gateway enforces the same precondition across all of them, with no per-contract tuning and without turning away the legitimate users those contracts still serve. The cost is modest gas and a round trip well inside the interval a transaction already waits before confirmation.

The two systems fit together, and that is where I want to spend the next five years. SAF treats its policy oracle as a replaceable component, and TxSentinel is an oracle needing no knowledge of

the protocol it protects. Combined, they defend a contract nobody has onboarded, on a chain nobody has modeled, and stop the attack at execution instead of reporting it afterwards.

The Wider Agenda

A solo position paper sets out the questions I consider still open [8], and three projects take them forward. In **provable smart-contract repair** I synthesize patches automatically and verify them with SMT solvers, so they are gas-aware and provably preserve intended behavior. **Adversarially robust detection** follows from the prevention work: once a detector gates execution it becomes worth attacking, so I combine certified-robust embeddings with adversarially regularized temporal graph networks. **Cross-chain contagion simulation** measures how exploits spread between chains, the layer regulators currently lack.

A fourth project takes the work into governance. **RWA tokenization with regulation enforcement** couples on-chain monitoring with machine-checkable compliance rules for custody, transfer restrictions, and investor eligibility, so tokenized assets enforce regulation continuously rather than at audit intervals. Security Authorization already provides the mechanism: the same guard that refuses an exploit can refuse a non-compliant transfer.

Future Plan: Years 1–3

At United Arab Emirates University I will build a research group in trustworthy decentralized systems around three funded thrusts, each scoped for regional grant programs (ASPIRE, the Technology Innovation Institute, and the UAE University Program for Advanced Research):

1. **Deployable prevention, not just detection.** I will harden TxSentinel and SAF into an auditable protection service for virtual-asset service providers and evaluate it with regional exchanges and custodians on the chains they actually use. The measurable goal is a protocol a licensed operator can adopt without restructuring its contracts.
2. **Compliance-by-construction tokenization.** I will formalize Gulf virtual-asset rulebooks (e.g., VARA and the Central Bank of the UAE frameworks) into machine-checkable policies and enforce them at the contract layer through the same authorization mechanism, in direct support of UAE Centennial 2071 digital-economy goals.
3. **Robust and certified AI for security.** A learned model that gates execution must resist evasion and justify its verdicts. I will extend my adversarial benchmarks into a certification methodology for the AI systems that govern financial infrastructure.

Each thrust yields M.Sc. and Ph.D. topics, top-venue publications, and open artifacts. I will recruit and train local students from day one, as I already do at NYUAD.

Future Plan: Years 3–5

I will widen the program from digital assets to the region’s critical digital infrastructure: security for central bank digital currency and settlement pilots, ledger-backed supply-chain and smart-city systems, and national-scale benchmarking carried out with sector regulators and CERTs. What I am working toward is a research center for decentralized-systems security that is based in the region and taken seriously outside it, and that trains the cybersecurity workforce these national programs will need.

References

- [1] Tamer Abdelaziz and Aquinas Hobor. Smart learning to find dumb contracts. In *32nd USENIX Security Symposium*, pages 1775–1792, 2023.
- [2] Tamer Abdelaziz and Aquinas Hobor. Schooling to exploit foolish contracts. In *5th Int. Conf. on Blockchain Computing and Applications (BCCA)*, pages 388–395, 2023.
- [3] Tamer Abdelaziz. *Towards Secure Smart Contracts: A Deep Learning Approach for Detecting Security Threats*. PhD thesis, National University of Singapore, 2023. ScholarBank@NUS: 10635/247301.
- [4] Tamer Abdelaziz, Salma Alsaghir, and Karim Ali. Where do smart contract security analyzers fall short? In *23rd Int. Conf. on Mining Software Repositories (MSR)*, 2026.
- [5] Tamer Abdelaziz and Karim Ali. TxLens: Scalable real-time detection of malicious Ethereum transactions. In *8th IEEE Int. Conf. on Blockchain and Cryptocurrency (ICBC)*, 2026.
- [6] Tamer Abdelaziz and Karim Ali. TxSentinel: Protocol-agnostic exploit detection on EVM blockchains. Under review, 2026.
- [7] Tamer Abdelaziz and Karim Ali. Contract-enforced security authorization: Stopping DeFi exploits before they execute. Under review, 2026.
- [8] Tamer Abdelaziz. Smart contract security beyond detection. arXiv preprint arXiv:2605.09124, 2026.